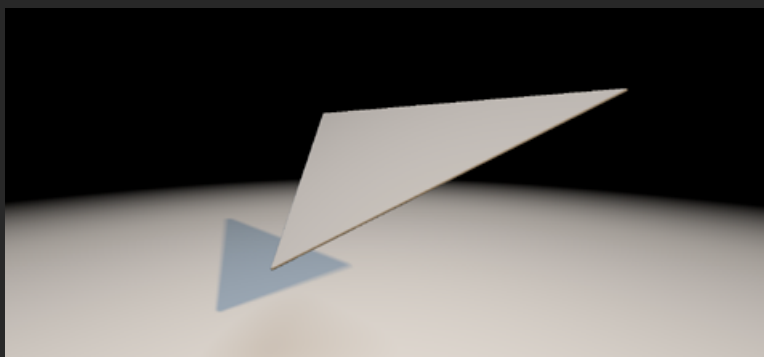


Inigo Quilez :: articles :: distance to triangle - 2014



Intro

Computing the distance to a triangle in 3D is not difficult, but often I've seen too convoluted code, with too many square roots and branches. Here goes my implementation, which I like and have used in raymarching experiments in Shadertoy.



Raymarched triangle, with the analytic distance computation

The distance is computed as the distance to the edges of the triangle or the distance to the plane of the triangle, depending where the point of evaluation lays with respect to the triangle. If the point is inside the prism you get from extruding the triangle across the direction of its normal, then the closest distance from the point to the triangle is the distance to the plane where the triangle exists. Otherwise, if the point is outside, the closest distance will be the distance to the closest of the three edges.

Computing the distance to a plane is easy, it's just the dot product of the point minus a corner of the triangle and the normal of the plane (normalized). Like $d = \text{dot}(p - v_1, \text{nor}) / \text{length}(\text{nor})$, if v_1 , v_2 and v_3 were the vertices of the triangle (nor would be the cross product of two edges, as usual, for example $v_2 - v_1$ and $v_1 - v_3$).

The distance to the edges can be computed by simply projecting the point relative to a vertex in the edge into the edge itself, and making sure the projection never goes below the first vertex or beyond the second. For example, for the edge going from vertex v_1 to vertex v_2 , the distance to our point p would be $d = \text{length}(p - v_1)$

```
- v21*clamp( dot(p-v1,v2-v1)/dot(v2-v1,v2-v1), 0, 1 ) ).
```

Before we can put both expressions together, we need to determine if our point is inside or outside of the triangle's extruded prism. We can do that by seeing if the point is in the inside or outside half space of each edge. The sign of the $\text{dot}(\text{cross}(v2-v1, \text{nor}), p2-v1)$ will for example measure this (you can think of this as a determinant or volume, or as the comparison between the direction of the normal and the cross product of our point-to-vertex and edge). If the three comparisons give a positive number, then we are inside the triangle. If any of the comparisons has negative sign, we are not.

A way to quickly check that the three comparisons are positive without having to do three comparisons, is to add the three results first. If all results are negative, the sum will be -3. If one is positive, the sum will be -1. If two are positive, the sum will be 1. And if all three are positive, the sum will be 3. So comparing the sum to 2 will separate our two cases - all inside or not.

With all this information, we can now compute our distance to a triangle in a more or less optimized way:

```
float dot2( in vec3 v ) { return dot(v,v); }

float udTriangle( in vec3 p, in vec3 v1, in vec3 v2, in vec3 v3 )
{
    // prepare data
    vec3 v21 = v2 - v1; vec3 p1 = p - v1;
    vec3 v32 = v3 - v2; vec3 p2 = p - v2;
    vec3 v13 = v1 - v3; vec3 p3 = p - v3;
    vec3 nor = cross( v21, v13 );

    return sqrt( // inside/outside test
        (sign(dot(cross(v21,nor),p1)) +
         sign(dot(cross(v32,nor),p2)) +
         sign(dot(cross(v13,nor),p3)) < 2.0)
        ?
        // 3 edges
        min( min(
            dot2(v21*clamp(dot(v21,p1)/dot2(v21),0.0,1.0)-p1),
            dot2(v32*clamp(dot(v32,p2)/dot2(v32),0.0,1.0)-p2) ),
            dot2(v13*clamp(dot(v13,p3)/dot2(v13),0.0,1.0)-p3) )
        :
        // 1 face
        dot(nor,p1)*dot(nor,p1)/dot2(nor) );
}
```

Note that much of this data could be precomputed if needed such as $v21$, $v32$, $v12$, nor , $\text{cross}(v21, \text{nor})$,

`cross(v32,nor), cross(v13,nor), 1.0/dot2(v21), 1.0/dot2(v32), 1.0/dot2(v13) and 1.0/dot2(nor).`

Also, note that I moved the square root needed for the edge distances outside of the branch. This has the advantage that if we were computing the distance to a mesh, which is made of many triangles, we could move the square root to an even more global scope and iterate all the triangles involved without square roots, take the minimum of the squared distance, and perform the square root only once at the very end.

This same code is listed in the page for [distance functions in this web site](#).

Also, there's a live example in Shadertoy of this code: <https://www.shadertoy.com/view/4sXXRN>

[inigo quilez](#) - learning computer graphics since 1994